



SynapseFS

Introduction

While fine-tuning large models (like 70B parameters) produces hundreds of gigabytes of serialized checkpoints, **for the scope of this problem statement, solutions will be built and evaluated on smaller models (up to 7b parameters) to accommodate standard hardware constraints.** Every existing tool which is used for this, be it Git LFS, HuggingFace Hub, S3, they treat each checkpoint as a flat binary blob and store full or near-full copies of each version.

There are two problems here:

1. Weights are **IEEE-754** floats, so non-trivial update to a value flips some of its bytes, i.e. a routine fine-tuning step touches some byte of the file and plain binary diff tools find very less in common even when the change was numerically small.
2. The second one is **Permutation Invariance**, i.e. swapping neuron indices in layer l and applying the corresponding permutation to layer $L+1$ yields an identical mathematical function, even if every byte in the checkpoint file changes position. Two checkpoints that compute the exact same function can look 100% different at every byte offset.

Task / Problem Statement

Build **SynapseFS**: a permutation-aware, cryptographically verifiable version control system and virtual filesystem for neural network checkpoints. Your system must expose intuitive Git-like operations (**commit**, **checkout**, **push**, **pull** and **merge**) while storing near-identical checkpoints as tiny, high-performance deltas.

Format: Checkpoints must be stored and read in **.safetensors** format; this is the required format for grading. Supporting legacy PyTorch **.pt/.bin** pickle checkpoints as an additional input format is optional and considered bonus effort - not required for the core deliverable.

Architecture Scope: Solutions must support alignment and versioning for standard MLPs and CNN architectures (e.g., ResNet-style blocks with convolutional, batch-norm/layer-norm, and linear layers). Transformer-based architectures (e.g.,



Takneek PS - High Prep



attention blocks, RoPE, SwiGLU) are not required and will not be part of the graded evaluation fixtures.

Modules:

1. Isomorphic Alignment Engine

- a. Given two checkpoints of the same architecture, figure out the correspondence between their hidden neurons.
- b. **Metadata & Topology:** Because .safetensors operates as a flat key-value store, evaluated checkpoints will be accompanied by standard `config.json` files defining the graph topology. Teams should design their topology parser against this general shape.
- c. Neuron correspondence covers: row/column permutations of weight matrices, matching channel/filter indices for convolutional layers, and the corresponding permutation of bias and normalization parameters (batch-norm/layer-norm affine terms). This applies to the MLP and CNN architectures defined in the Architecture Scope above.
- d. **Permutation Symmetries:** In cases where neuron permutation is non-unique (e.g., identically initialized weights), any mathematically valid permutation that preserves the network's function and reconstructs the target tensor is acceptable.
- e. Using that correspondence, produce a compact artifact capturing only the genuine functional difference between the two checkpoints. It should stay small when the models are semantically close, regardless of how their neurons happen to be indexed on disk.
- f. The diff artifact must be self-sufficient: given it plus one of the two checkpoints, you must be able to reconstruct the other exactly. **"Exactly" means the reconstructed file must be byte-for-byte identical to the target .safetensors file, including all headers and metadata.**
- g. **"Self-sufficient"** means the diff artifact, together with one checkpoint, reconstructs the other; it does not mean alignment must be re-inferred at reconstruction time. The diff artifact is permitted (and expected) to store the permutation/correspondence mapping alongside the compressed residuals.
- h. Correctly recognize when two checkpoints are not meaningfully alignable and report that explicitly, rather than forcing a low-confidence match and displaying it as a result.
- i. All of the above must hold consistently across checkpoints of meaningfully different scale within the **50M–7B parameter range** used



Takneek PS - High Prep



for evaluation. Alignment robustness should be demonstrated on both a small and a large model within the same architecture family (for example a 100M and a 7B-parameter variant), not just one size you happened to tune for. You are not required to align checkpoints of different total parameter counts against each other (e.g., a 1B against a 7B); that is a distinct problem outside this PS's scope.

For checkpoints that don't fit in the stated RAM budget, alignment must be performed out-of-core.

2. Verifiable Storage and Lineage

- a. Every storage checkpoint must be cryptographically verifiable, i.e. any tampering with any part of the stored history must be reliably detected.
- b. **Threat Model:** The system must utilize content-addressable storage (e.g., Merkle DAGs) to detect and reject both accidental file corruption (bit-rot) and malicious block injection (e.g., from a compromised peer). Implementing user authentication or GPG commit signing is not required.
- c. Trust is rooted at a locally accepted commit/ref ID – content addressing must detect any mismatch between stored bytes and their expected hash under that lineage. Defending against a peer presenting an entirely different but internally self-consistent history (ref rollback, wholesale history replacement) is out of scope.
- d. Verifying a checkpoint's full lineage must be fast even at multi-gigabyte scale.
- e. The system must retain enough lineage information to reconstruct any previously stored checkpoint version exactly.
- f. The system must support branching (two or more commits sharing the same parent) and correctly walk and verify any branch, not only a single linear chain.
- g. The system must support merging two branches by reconciling their commit DAGs (a standard three-way or fast-forward merge over lineage/history, analogous to `git merge`).
- h. A crash or forced kill of the storage process mid-write must never leave the on-disk history corrupted or unverifiable. On restart, the system must either recover cleanly or **safely refuse to proceed**.
The crash model covers process termination on a standard local Linux filesystem.

3. Transparent Filesystem Access Layer

- a. Target OS for development and evaluation: Linux only. FUSE and mmap behavior are not required to work on macOS or native Windows.



Takneek PS - High Prep



- b. Expose reconstructed checkpoints through a POSIX-compliant virtual mount supporting `read()/mmap()`, so `safetensors.torch.load_file()` (or `transformers.AutoModel.from_pretrained()` with a `safetensors` checkpoint) works unmodified against it. FUSE might be a fit for this, but teams are open to decide on mechanisms.
 - c. **Read-Only Scope:** The virtual mount is strictly read-only. POSIX compliance will only be evaluated on system calls necessary for unmodified ML libraries to read data (e.g., `open`, `read`, `mmap`, `lseek`, `stat`, `close`).
 - d. Reconstruct requested tensor regions on-the-fly from base + residual blocks.
 - e. **Strictly No Pre-materialization:** You are forbidden from reconstructing and saving the entire checkpoint to the disk as a temporary file upon mounting. Tensors must be constructed dynamically in RAM/VRAM as page faults or reads occur. Benchmarking will be tested from a cold OS page cache.
 - f. Multiple concurrent readers accessing the mount must all get correct data, with no crashes and no corruption from data races.
4. **Distributed Networking**
- a. Two peer instances must be able to synchronize checkpoint history with each other, transferring only the data blocks a peer doesn't already have, instead of a full checkpoint on every sync.
 - b. A sync interrupted partway must leave both peers in a recoverable state. Resuming afterward must not require re-transferring data already received, and must never corrupt either peer's history.
 - c. Teams may use any transport layer (HTTP, gRPC, raw sockets, etc.) - the transport itself is not graded. What is graded is that content-addressed block-diffing (determining which blocks a peer is missing) is implemented by the team, not delegated to an existing sync tool (e.g., `rsync`, `rclone`).

Consistency Requirement

A checkpoint restored through your tool's checkout/restore capability and the same checkpoint read through the virtual mount must be byte-identical once reconstructed, regardless of how either path is implemented.



Takneek PS - High Prep



Deliverables

Source Repository

- Full commit history.
- Build instructions from a clean environment (no assumed pre installed toolchain beyond what's documented).
- Keep large synthetic test fixtures out of the commit history where you can. You can script their generation instead of committing them.

CLI Binary

A binary (or documented one-command build) implementing `init`, `commit`, `checkout`, `branch`, `log`, `verify`, `push`, `pull`, `serve`, `merge`, `mount` and `unmount`.

- `branch/checkout`: `checkout` restores a specific checkpoint version to the working state; `branch` creates or lists named branches; switching the active branch is done via `checkout <branch-name>`, consistent with pre-2.23 Git semantics.
- `verify`: walks and cryptographically verifies lineage for the current branch (or a specified commit), independent of `checkout/mount`, so integrity can be graded standalone.
- `serve`: starts a peer's sync-listening process so another instance can push/pull against it; document the invocation (address/port/config) in the README.

Clear and precise documentation, hosted, explaining:

- Project architecture and setup instructions from scratch.
- The alignment algorithm and storage format design.
- Trade-offs considered and why the chosen approach was picked.

Presentation

- Maximum 10 minutes.
- At least 2 members will be presenting.

ZIP file

Should contain all source code, matching the GitHub repo submission exactly (with `.git` folder).

Deadline: 2nd September 2026, 2200 hrs.



Evaluation Metrics

1. Alignment & Compression (25%)
 - a. Permutation-recovery accuracy (10%)
 - i. How accurately the system recovers permutations against various unseen checkpoint test pairs of varying architectures and parameter sizes.
 - ii. **Constraint Note:** Automated evaluation fixtures will cap model sizes at roughly 7B parameters. Evaluated systems must be able to perform this alignment and generate the residual artifact entirely within standard consumer hardware limits (Maximum 16GB System RAM, 8GB VRAM). Solutions that crash with Out-Of-Memory (OOM) errors on these test sizes will fail this metric.
 - iii. Checkpoints will be evaluated in fp16/bf16 precision (not fp32). For checkpoints whose full in-memory footprint would exceed the stated RAM/VRAM budget, alignment must be performed out-of-core, i.e. streaming corresponding layer pairs from disk sequentially (e.g., via safetensors' lazy tensor access) rather than loading the entire checkpoint into memory at once. Solutions that only work by loading full checkpoints into RAM, and would OOM at the stated fixture sizes, will fail this metric even if they pass on a smaller local test.
 - b. Alignment wall-clock time (8%)
 - i. The computational efficiency and wall-clock time required to align weights on standard evaluation hardware.
 - c. Residual ratio (7%)
 - i. The achieved residual ratio (the size of the generated residual blocks relative to the original checkpoint size).
2. Filesystem Access & Memory (25%)
 - a. POSIX compliance (10%)
 - i. How seamlessly the virtual mount handles standard file operations and edge cases for unmodified ML libraries including correctness under multiple concurrent readers accessing the mount simultaneously.
 - b. `mmap` read throughput (8%)
 - i. The performance and latency of `mmap` and read operations when dynamically reconstructing checkpoints under load.
 - c. Daemon peak RSS (7%)
 - i. The daemon's memory footprint (Peak RSS) and CPU utilization during sustained I/O tasks.



Takneek PS - High Prep



3. Cryptographic Integrity (20%)
 - a. Tamper detection (10%)
 - i. The system's ability to reliably detect and reject tampered blocks across different attack scenarios while validating intact DAG chains.
 - b. Verification time (10%)
 - i. The speed at which the system can walk the DAG and verify the lineage for large, multi-gigabyte reconstructed models.
4. Networking & CLI (15%)
 - a. Functional Reliability (10%)
 - i. The consistency and correct execution of CLI operations (init, commit, checkout, branch, log, merge, mount, unmount) across different repository states, including recovery behavior after a simulated crash mid-write.
 - b. Differential Transfer (5%)
 - i. The efficiency of the push and pull network mechanisms – specifically ensuring that only missing blocks are transferred during partial updates, and that a sync interrupted partway can resume correctly without re-transferring already-received blocks or corrupting either peer's history.
5. Documentation and Presentation (15%)
 - a. Clarity and completeness of documentation (5%)
 - i. System architecture: Explaining the alignment algorithm, storage format, and tech stack choices.
 - ii. Trade-offs: Justifying the specific alignment approximation or compression scheme used over alternative approaches.
 - b. A well-structured README file (5%)
 - i. Clear instructions for project setup from scratch and a functional testing script/harness.
 - c. Presentation (5%)
 - i. Maximum of 10 minutes. At least 2 members present, heavily graded on their ability to defend code and architectural choices during the Q&A.

Note: The weightage next to each component is not used for objective scoring, it only gives participants a sense of relative importance. All scoring, including these weightages, is subjective, judgement based, and subject to change. For the architectural components in particular (alignment, storage/lineage, filesystem access, networking), judges will favor solutions that show original thinking, well-reasoned trade-offs, and a deep understanding of the problem



Takneek PS - High Prep



over solutions that simply follow the most common or first-suggested approach. Two systems that "work" equally well on the surface may score differently if one handles edge cases more smartly, justifies its method choices, or tries something non-obvious that improves cost, accuracy, or speed. Teams are strongly encouraged to explain why they built something a certain way, not just what they built, in both documentation and presentation, since judges will factor this reasoning directly into scores.

Guidelines:

Strict alterations in evaluation procedures will be taken in order to judge the problem statement on the merits of the developers and not the merits of the Large Language Model or any vibe coding. These shall, again be highly subject to the experience of the evaluator, but we shall try to list down the same in a structured fashion:

1. **Focus on knowledge during presentation:** If developers are not able to answer any question on a feature or module during the presentation, but the same is present in the codebase, the implementation of the feature shall be considered null and void.
2. **Clear description and understanding of system architectures and tradeoff considerations:** Large Language Models often come up with standard procedures, use standard tools, program functionalities in a certain fashion. Think beyond the suggestion of the LLM, research about various optimizations, understand tradeoffs and do research from ground up. Read blogs, documentations, understand why certain tools are preferred over the others in certain scenarios etc. These explanations should also be clearly mentioned in the README, and the participants should have a detailed understanding of these explanations. If we find that the explanations are AI-generated and participants are not able to answer questions, heavy penalties will be applied.
3. Lastly, even if the quality of one solution is objectively better than the other - we might choose to judge the objectively worse solution as the better one in case the developers of the prior are not able to explain their solutions.



Takneek PS - High Prep



On involvement of Y26s: In past competitions, we have observed a recurring issue where freshers do not actively participate in problem solving and are merely coached on specific sections for the presentation. To prevent this deceptive practice, presentations will now feature two distinct Q&A sessions: a general session for the entire team, and a dedicated session exclusively for Y26 students. The Y26 Q&A will go beyond generic concepts to test specific, implementation-based knowledge. If it becomes evident that the freshers did not actively contribute to building the project from scratch, the team will face severe penalties. We shall not reveal more on the exact pattern of the Y26 Q&A but broadly they will be tested on their fundamentals, contribution and participation in the PS.

Team Composition:

Each team should have **8 members**, with **at least 3 Y26s** and **3 Y25s**.

For Any Queries, The Pool Captains and PS Leads are encouraged to ask in the Discord channel.

